

API Gateway in Microservices Architecture

A Comprehensive Guide with Spring Cloud Gateway

Table of Contents

1. Problems with Direct Client Access
2. Role of an API Gateway
3. Benefits of API Gateway
4. Request Routing and Path Rewriting
5. Gateway Filters
6. Load Balancing Concept
7. Simplifying Client-Side Communication
8. When an API Gateway is Essential
9. Real-world Architecture Example

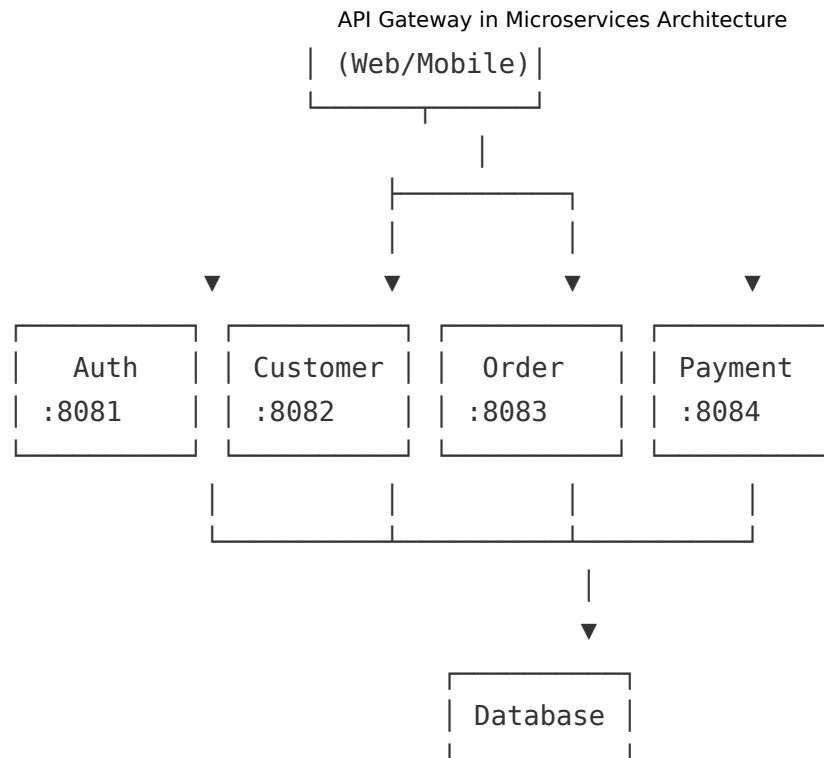
1. Problems with Direct Client Access

The Challenge

In a microservices architecture without an API Gateway, clients (web browsers, mobile apps, third-party applications) must directly communicate with multiple microservices. Each service exposes its own API on different ports and endpoints.

Architecture Without API Gateway





Problems Identified

1. Multiple Service Endpoints

Clients need to know and manage multiple URLs:

- Authentication: `http://localhost:8081/api/auth`
- Customers: `http://localhost:8082/api/customers`
- Orders: `http://localhost:8083/api/orders`
- Payments: `http://localhost:8084/api/payments`
- Restaurants: `http://localhost:8085/api/restaurants`
- Menus: `http://localhost:8086/api/menus`
- Delivery Partners: `http://localhost:8087/api/delivery-partners`
- Notifications: `http://localhost:8088/api/notifications`

Impact:

- Complex client configuration
- Difficult to maintain across environments
- CORS issues with multiple origins
- SSL certificate management for each service

2. Cross-Cutting Concerns Duplication

Each microservice must implement:

- Authentication and authorization
- Request validation
- Logging and monitoring
- Rate limiting
- CORS handling

Impact:

- Code duplication across services
- Inconsistent implementation
- Maintenance overhead
- Security risks (inconsistent security)

3. Client Complexity

Clients must handle:

- Service discovery (finding service URLs)
- Load balancing (distributing requests)
- Retry logic (handling failures)
- Circuit breaking (preventing cascading failures)
- API versioning (managing different service versions)

Impact:

- Complex client code
- Business logic mixed with infrastructure concerns
- Difficult to test and maintain
- Poor developer experience

4. Security Concerns

Direct access exposes:

- Internal service structure to clients
- Multiple attack surfaces
- Difficult to implement consistent security policies
- No centralized authentication/authorization

Impact:

- Increased attack surface
- Inconsistent security policies
- Difficult to audit and monitor
- Compliance challenges

5. Performance Issues

Without optimization:

- Multiple network calls from client
- No request/response caching
- No request/response compression
- No request aggregation

Impact:

- Slower response times
- Higher bandwidth usage
- Poor user experience
- Increased infrastructure costs

6. Operational Challenges

Operations team must manage:

- Multiple service endpoints
- Different authentication mechanisms
- Separate monitoring for each service
- Complex deployment strategies

Impact:

- Increased operational complexity
- Difficult troubleshooting
- Longer incident response times
- Higher operational costs

Real-world Example

Without API Gateway - Client Code:

```
// Client must know all service URLs
const AUTH_SERVICE = 'http://localhost:8081';
const CUSTOMER_SERVICE = 'http://localhost:8082';
const ORDER_SERVICE = 'http://localhost:8083';
const PAYMENT_SERVICE = 'http://localhost:8084';

// Client must implement load balancing
async function getCustomer(id) {
    const instances = await discoverInstances('customer-service');
    const instance = selectInstance(instances);
    return fetch(`${instance}/api/customers/${id}`);
}

// Client must implement retry logic
async function createOrder(order) {
    let retries = 3;
    while (retries > 0) {
        try {
            const response = await fetch(`${ORDER_SERVICE}/api/orders`, {
                method: 'POST',
                body: JSON.stringify(order)
            });
            return response.json();
        } catch (error) {
            retries--;
            if (retries === 0) throw error;
        }
    }
}
```

2. Role of an API Gateway

Definition

An API Gateway is a server that acts as an API front-end, receiving API requests, enforcing throttling and security policies, passing requests to the appropriate service, and returning the

appropriate response. It's the single entry point for all client requests in a microservices architecture.

Core Responsibilities

1. Request Routing

- Route requests to appropriate microservices based on URL patterns
- Support multiple routing strategies (path-based, header-based, method-based)
- Handle service discovery integration

2. Protocol Translation

- Convert between different protocols (HTTP, WebSocket, gRPC)
- Support multiple client types (web, mobile, IoT)
- Handle API versioning

3. Request/Response Transformation

- Modify requests before forwarding to services
- Transform responses before returning to clients
- Aggregate responses from multiple services

4. Security Enforcement

- Authentication and authorization
- JWT token validation
- OAuth/OIDC integration
- API key management

5. Cross-Cutting Concerns

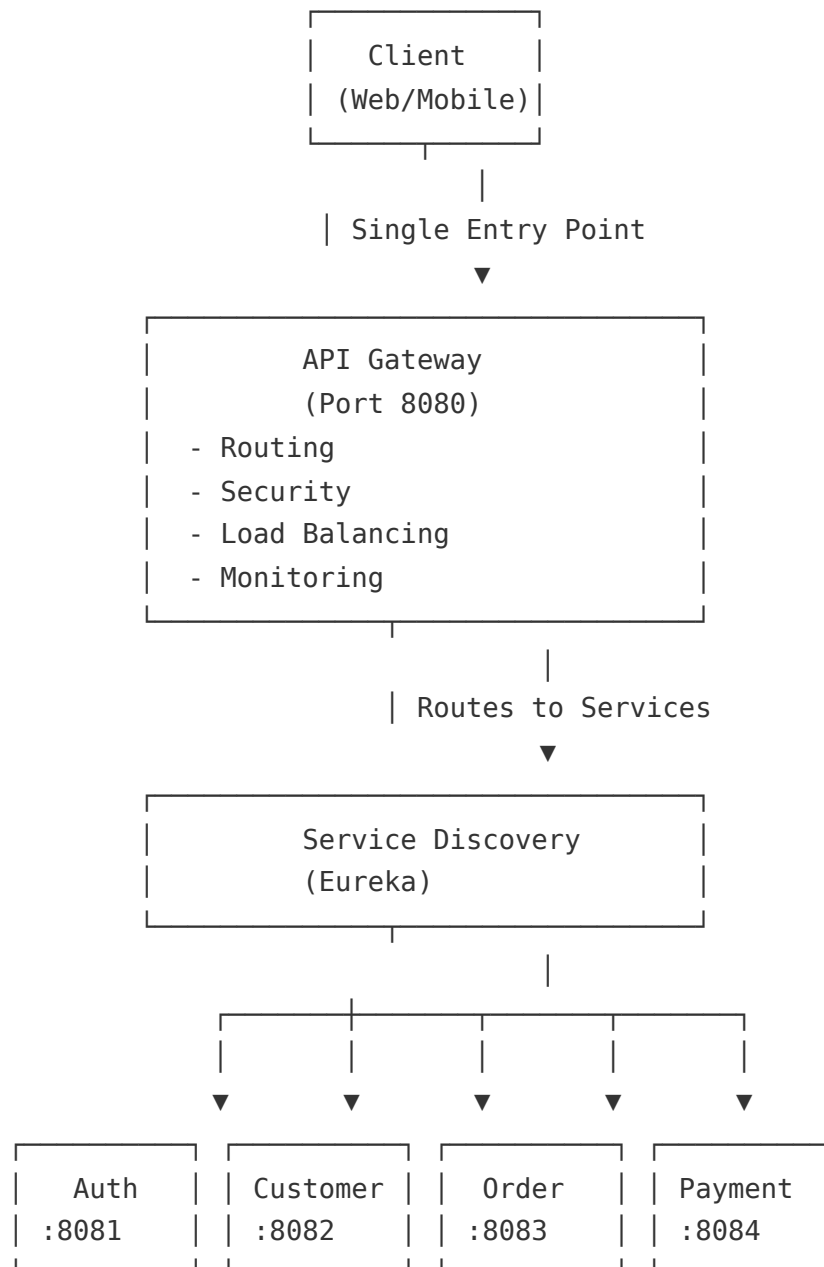
- Logging and monitoring
- Rate limiting
- Caching
- CORS handling
- Request validation

6. Load Balancing

- Distribute requests across service instances
- Implement various load balancing strategies

- Handle failover and retries

Architecture with API Gateway



Key Characteristics

1. Single Entry Point

- All client requests go through the gateway
- Simplifies client configuration

- Centralized access control

2. Decoupling

- Clients don't need to know about internal service structure
- Services can evolve independently
- Reduces client complexity

3. Abstraction Layer

- Hides service implementation details
- Provides unified API interface
- Enables backend changes without client impact

4. Centralized Management

- Single place for cross-cutting concerns
- Consistent policy enforcement
- Simplified monitoring and debugging

Types of API Gateways

1. General-Purpose API Gateways

- Kong
- NGINX
- HAProxy
- Traefik

2. Cloud-Native API Gateways

- AWS API Gateway
- Azure API Management
- Google Cloud Endpoints

3. Framework-Specific Gateways

- Spring Cloud Gateway (Java/Spring)
- Express Gateway (Node.js)
- Zuul (Netflix - deprecated)

4. Service Mesh Gateways

- Istio Gateway
- Linkerd
- Consul Connect

3. Benefits of API Gateway

1. Single Entry Point

Description:

All client requests are routed through a single endpoint, simplifying client configuration and reducing complexity.

Benefits:

- **Simplified Client Configuration:** Clients only need to know one URL
- **Unified API Surface:** Consistent API interface across all services
- **Easier Testing:** Single endpoint to test integration
- **Better Developer Experience:** Clear, documented entry point

Example:

Before: Multiple URLs

- `http://localhost:8081/api/auth`
- `http://localhost:8082/api/customers`
- `http://localhost:8083/api/orders`

After: Single URL

- `http://localhost:8080/api/auth`
- `http://localhost:8080/api/customers`
- `http://localhost:8080/api/orders`

Our Implementation:

```
server:
  port: 8080 # Single entry point for all services

spring:
  application:
    name: api-gateway
```

2. Centralized Routing

Description:

All routing logic is centralized in the gateway, making it easy to manage and modify routing rules.

Benefits:

- **Easy Configuration:** Routing rules in one place
- **Dynamic Updates:** Routes can be updated without code changes
- **Version Management:** Support for API versioning
- **A/B Testing:** Route traffic to different service versions

Example:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
```

Routing Strategies:

- **Path-based:** Route based on URL path
- **Header-based:** Route based on request headers
- **Method-based:** Route based on HTTP method

- **Query parameter-based:** Route based on query parameters

3. Security Enforcement

Description:

Centralized security policies ensure consistent authentication and authorization across all services.

Benefits:

- **Consistent Security:** Same security rules for all services
- **Reduced Attack Surface:** Only gateway needs to be exposed publicly
- **Centralized Authentication:** Single authentication mechanism
- **Easier Compliance:** Easier to audit and enforce security policies

Security Features:

- **Authentication:** JWT validation, OAuth/OIDC
- **Authorization:** Role-based access control
- **Rate Limiting:** Prevent abuse and DDoS attacks
- **IP Whitelisting/Blacklisting:** Control access by IP
- **Request Validation:** Validate requests before forwarding

Example Configuration:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
            - AddRequestHeader=X-Request-Id, ${requestId}
```

4. Request Filtering

Description:

Gateway can filter and modify requests before they reach backend services.

Benefits:

- **Request Validation:** Validate requests before forwarding
- **Header Manipulation:** Add, modify, or remove headers
- **Request Transformation:** Modify request body or parameters
- **Logging:** Log all requests for monitoring and debugging

Filter Types:

- **Pre-filters:** Execute before routing
- **Post-filters:** Execute after routing
- **Error filters:** Execute on errors

Example:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
            - AddRequestHeader=X-Gateway-Request, true
            - AddResponseHeader=X-Gateway-Response, true
```

5. Rate Limiting (Concept Overview)

Description:

Rate limiting controls the number of requests a client can make within a specific time period.

Benefits:

- **Prevent Abuse:** Protect services from malicious attacks
- **Fair Usage:** Ensure fair resource allocation
- **Cost Control:** Manage infrastructure costs
- **SLA Enforcement:** Ensure service level agreements

Rate Limiting Strategies:

- **Fixed Window:** Limit requests per fixed time window
- **Sliding Window:** More accurate rate limiting
- **Token Bucket:** Allow bursts of requests
- **Leaky Bucket:** Smooth out request rate

Implementation Approaches:

- **Gateway-level:** Apply at gateway for all services
- **Service-level:** Apply per service
- **User-level:** Apply per user/IP
- **Endpoint-level:** Apply per API endpoint

Example (Spring Cloud Gateway with Redis):

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
```

6. Load Balancing

Description:

Gateway distributes requests across multiple service instances to ensure optimal resource utilization.

Benefits:

- **High Availability:** Distribute load for better performance
- **Fault Tolerance:** Route away from failed instances
- **Scalability:** Handle increased traffic
- **Resource Optimization:** Better resource utilization

Load Balancing Strategies:

- **Round Robin:** Distribute requests sequentially
- **Random:** Randomly select instances
- **Weighted:** Distribute based on instance capacity
- **Least Connections:** Route to instance with fewest connections

Example:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service # Load balanced
          predicates:
            - Path=/api/orders/**
```

7. Monitoring and Logging

Description:

Centralized logging and monitoring provide visibility into all API traffic.

Benefits:

- **Troubleshooting:** Easy to debug issues
- **Performance Monitoring:** Track API performance
- **Analytics:** Understand usage patterns
- **Audit Trail:** Track all API calls

Monitoring Capabilities:

- Request/response logging
- Performance metrics (latency, throughput)
- Error tracking
- Custom metrics

Example:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,gateway
```

8. API Versioning

Description:

Gateway can manage multiple API versions, allowing smooth transitions.

Benefits:

- **Backward Compatibility:** Support old and new versions
- **Smooth Migration:** Gradual rollout of new versions
- **Client Choice:** Clients can choose their version
- **Deprecation Management:** Manage API lifecycle

Example:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-v1
          uri: lb://order-service
          predicates:
            - Path=/api/v1/orders/**
          filters:
            - StripPrefix=2
        - id: order-service-v2
          uri: lb://order-service
          predicates:
            - Path=/api/v2/orders/**
          filters:
            - StripPrefix=2
```

4. Request Routing and Path Rewriting

Request Routing

Definition:

Request routing is the process of determining which backend service should handle a client request based on predefined rules.

Routing Predicates

Spring Cloud Gateway uses predicates to match incoming requests:

1. Path Predicate

```
predicates:
  - Path=/api/orders/**
```

Matches requests with path starting with `/api/orders/`

2. Method Predicate


```
predicates:  
  - Method=GET,POST
```

Matches GET and POST requests

3. Header Predicate

```
predicates:  
  - Header=X-Request-Id, \d+
```

Matches requests with specific header pattern

4. Query Predicate

```
predicates:  
  - Query=category
```

Matches requests with query parameter

5. Host Predicate

```
predicates:  
  - Host=api.example.com
```

Matches requests from specific host

6. Cookie Predicate

```
predicates:  
  - Cookie=session, \w+
```

Matches requests with specific cookie

7. Time Predicate

```
predicates:  
  - Between=2024-01-01T00:00:00+00:00,2024-12-31T23:59:59+00:00
```

Matches requests within time range

Path Rewriting

Definition:

Path rewriting modifies the request path before forwarding it to the backend service.

Common Path Rewriting Filters

1. StripPrefix Filter

Removes specified number of path segments before forwarding.

```
filters:
  - StripPrefix=1
```

Example:

- Incoming: `http://gateway:8080/api/orders/123`
- After StripPrefix=1: `http://order-service:8085/orders/123`

2. RewritePath Filter

Rewrites path using regex pattern.

```
filters:
  - RewritePath=/api/(?.*), /${segment}
```

Example:

- Incoming: `http://gateway:8080/api/orders/123`
- After RewritePath: `http://order-service:8085/orders/123`

3. SetPath Filter

Sets the path to a specific value.

```
filters:
  - SetPath=/api/orders/{id}
```

Our Implementation

Service Routes:

```
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=1
        - id: customer-service
          uri: lb://customer-service
          predicates:
            - Path=/api/customers/**
          filters:
            - StripPrefix=1
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
```

OpenAPI Aggregation Routes:

```
- id: openapi-auth-service
  uri: lb://auth-service
  predicates:
    - Path=/v3/api-docs/auth-service
  filters:
    - RewritePath=/v3/api-docs/auth-service, /v3/api-docs
```

Request Flow:

Client Request:

GET http://localhost:8080/api/orders/123

Gateway Processing:

1. Matches route: Path=/api/orders/**
2. Applies filter: StripPrefix=1
3. Resolves service: lb://order-service
4. Forwards to: http://order-service-instance/orders/123

Advanced Routing Examples

1. Header-based Routing

- id: order-service-v1
uri: lb://order-service
predicates:
 - Path=/api/orders/**
 - Header=X-API-Version, 1.0filters:
 - StripPrefix=1
- id: order-service-v2
uri: lb://order-service
predicates:
 - Path=/api/orders/**
 - Header=X-API-Version, 2.0filters:
 - StripPrefix=1

2. Method-based Routing

```
- id: order-service-get
  uri: lb://order-service
  predicates:
    - Path=/api/orders/**
    - Method=GET
  filters:
    - StripPrefix=1

- id: order-service-post
  uri: lb://order-service
  predicates:
    - Path=/api/orders/**
    - Method=POST
  filters:
    - StripPrefix=1
```

3. Weight-based Routing (A/B Testing)

```
- id: order-service-v1
  uri: lb://order-service
  predicates:
    - Path=/api/orders/**
    - Weight=group1, 80
  filters:
    - StripPrefix=1

- id: order-service-v2
  uri: lb://order-service
  predicates:
    - Path=/api/orders/**
    - Weight=group1, 20
  filters:
    - StripPrefix=1
```

5. Gateway Filters

Filter Types

Spring Cloud Gateway supports two types of filters:

1. Gateway Filters

- Applied to individual routes
- Modify request/response for specific routes
- Configured per route

2. Global Filters

- Applied to all routes
- Modify request/response globally
- Applied to every request

Built-in Gateway Filters

1. AddRequestHeader

Adds a header to the request.

```
filters:  
  - AddRequestHeader=X-Request-Id, ${requestId}
```

2. AddResponseHeader

Adds a header to the response.

```
filters:  
  - AddResponseHeader=X-Response-Time, ${responseTime}
```

3. RemoveRequestHeader

Removes a header from the request.

```
filters:  
  - RemoveRequestHeader=Cookie
```

4. SetRequestHeader

Sets a header value (overwrites if exists).

```
filters:  
  - SetRequestHeader=X-Forwarded-For, ${clientId}
```

5. SetResponseHeader

Sets a response header value.

```
filters:  
  - SetResponseHeader=X-Custom-Header, CustomValue
```

6. AddRequestParameter

Adds a parameter to the request.

```
filters:  
  - AddRequestParameter=version, v1
```

7. RemoveRequestParameter

Removes a parameter from the request.

```
filters:  
  - RemoveRequestParameter=debug
```

8. RedirectTo

Redirects to another URL.

```
filters:  
  - RedirectTo=302, /new-location
```

9. StripPrefix

Removes path segments.

```
filters:
  - StripPrefix=1
```

10. RewritePath

Rewrites path using regex.

```
filters:
  - RewritePath=/api/(?.*), /${segment}
```

11. RequestRateLimiter

Applies rate limiting.

```
filters:
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 10
      redis-rate-limiter.burstCapacity: 20
```

12. CircuitBreaker

Implements circuit breaker pattern.

```
filters:
  - name: CircuitBreaker
    args:
      name: orderServiceCircuitBreaker
      fallbackUri: forward:/fallback/orders
```

Custom Filters

You can create custom filters for specific requirements:

Example: Logging Filter


```
@Component
public class LoggingFilter implements GlobalFilter {

    private static final Logger log = LoggerFactory.getLogger(LoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        log.info("Request: {} {}", request.getMethod(), request.getURI());

        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            ServerHttpResponse response = exchange.getResponse();
            log.info("Response: {}", response.getStatusCode());
        }));
    }
}
```

Example: Authentication Filter

```
@Component
public class AuthenticationFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");

        if (token == null || !isValidToken(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        return chain.filter(exchange);
    }

    private boolean isValidToken(String token) {
        // Token validation logic
        return true;
    }
}
```

Filter Order

Filters are executed in a specific order:

1. Pre-filters (before routing)
2. Routing
3. Post-filters (after routing)

You can control filter order using `@Order` annotation:

```
@Component
@Order(1)
public class FirstFilter implements GlobalFilter {
    // ...
}

@Component
@Order(2)
public class SecondFilter implements GlobalFilter {
    // ...
}
```

Our Implementation

Current Filters in Use:

```
filters:
  - StripPrefix=1 # Remove /api prefix
  - RewritePath=/v3/api-docs/auth-service, /v3/api-docs # For OpenAPI
```

Additional Filters We Could Add:

```
filters:
  - AddRequestHeader=X-Gateway-Request, true
  - AddResponseHeader=X-Gateway-Response, true
  - AddRequestHeader=X-Request-Id, ${requestId}
  - AddResponseHeader=X-Response-Time, ${responseTime}
```

Logging Filter Example

Purpose: Log all requests and responses for monitoring and debugging.

Implementation:

```
@Component
@Slf4j
public class RequestLoggingFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        long startTime = System.currentTimeMillis();
        String requestId = UUID.randomUUID().toString();

        // Add request ID to exchange
        exchange.getAttributes().put("requestId", requestId);

        // Log request
        log.info("Request [{}]: {} {} from {}",
            requestId,
            exchange.getRequest().getMethod(),
            exchange.getRequest().getURI(),
            exchange.getRequest().getRemoteAddress());

        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            long duration = System.currentTimeMillis() - startTime;
            log.info("Response [{}]: Status {} in {}ms",
                requestId,
                exchange.getResponse().getStatusCode(),
                duration);
        })));
    }
}
```

Authentication Filter Example

Purpose: Validate JWT tokens before forwarding requests.

Implementation:

```
@Component
@Slf4j
public class JwtAuthenticationFilter implements GlobalFilter {

    @Value("${jwt.secret}")
    private String jwtSecret;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getPath().value();

        // Skip authentication for public endpoints
        if (isPublicEndpoint(path)) {
            return chain.filter(exchange);
        }

        String token = extractToken(exchange.getRequest());

        if (token == null) {
            return unauthorized(exchange);
        }

        if (!validateToken(token)) {
            return unauthorized(exchange);
        }

        // Add user info to headers
        exchange.getRequest().mutate()
            .header("X-User-Id", extractUserId(token))
            .build();

        return chain.filter(exchange);
    }

    private boolean isPublicEndpoint(String path) {
        return path.startsWith("/api/auth") ||
            path.startsWith("/swagger-ui") ||
            path.startsWith("/v3/api-docs");
    }
}
```

```
private String extractToken(ServerHttpRequest request) {
    String authHeader = request.getHeaders().getFirst("Authorization");
    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        return authHeader.substring(7);
    }
    return null;
}

private boolean validateToken(String token) {
    // JWT validation logic
    return true;
}

private String extractUserId(String token) {
    // Extract user ID from token
    return "123";
}

private Mono<Void> unauthorized(ServerWebExchange exchange) {
    exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
    exchange.getResponse().getHeaders().add("Content-Type", "application/json");
    String body = "{\"error\": \"Unauthorized\"}";
    DataBuffer buffer = exchange.getResponse().bufferFactory().wrap(body);
    return exchange.getResponse().writeWith(Mono.just(buffer));
}
}
```

6. Load Balancing Concept

What is Load Balancing?

Load balancing is the process of distributing incoming network traffic across multiple servers or service instances to ensure no single server bears too much demand.

Why Load Balancing is Important

1. High Availability

- Distributes traffic to prevent overload
- Ensures services remain available under high load
- Prevents single point of failure

2. Scalability

- Handles increased traffic by adding more instances
- Scales horizontally without client changes
- Optimizes resource utilization

3. Performance

- Reduces response times
- Improves user experience
- Maximizes throughput

4. Fault Tolerance

- Routes traffic away from failed instances
- Automatic failover
- Graceful degradation

Load Balancing Strategies

1. Round Robin

Distributes requests sequentially across instances.

```
Request 1 → Instance A  
Request 2 → Instance B  
Request 3 → Instance C  
Request 4 → Instance A
```

Pros:

- Simple to implement
- Fair distribution
- No state required

Cons:

- Doesn't consider instance load

- Doesn't consider instance capacity

2. Random

Randomly selects an instance for each request.

```
Request 1 → Instance B (random)
Request 2 → Instance A (random)
Request 3 → Instance C (random)
```

Pros:

- Simple implementation
- Good statistical distribution
- No state required

Cons:

- Doesn't consider instance load
- Potential uneven distribution

3. Weighted Round Robin

Distributes requests based on instance weights.

```
Instance A (weight 3): 3 requests
Instance B (weight 2): 2 requests
Instance C (weight 1): 1 request
```

Pros:

- Considers instance capacity
- Flexible distribution
- Good for heterogeneous instances

Cons:

- Requires weight configuration
- More complex than simple round robin

4. Least Connections

Routes to instance with fewest active connections.


```
Instance A: 5 connections  
Instance B: 3 connections ← Selected  
Instance C: 7 connections
```

Pros:

- Considers current load
- Better for long-running requests
- Dynamic adaptation

Cons:

- Requires connection tracking
- More complex implementation

5. IP Hash

Routes based on client IP hash.

```
Client IP 192.168.1.1 → Instance A  
Client IP 192.168.1.2 → Instance B  
Client IP 192.168.1.1 → Instance A (consistent)
```

Pros:

- Session persistence
- Consistent routing
- Good for stateful services

Cons:

- Uneven distribution with few clients
- Not ideal for load balancing

Load Balancing in API Gateway**Integration with Service Discovery:**

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service # Load balanced
          predicates:
            - Path=/api/orders/**
```

How it Works:

1. Gateway receives request
2. Resolves service name via Eureka
3. Gets list of available instances
4. Applies load balancing strategy
5. Routes to selected instance
6. Retries on failure

Spring Cloud Load Balancer

Spring Cloud Gateway uses Spring Cloud Load Balancer (replaces Ribbon):

Configuration:

```
spring:
  cloud:
    loadbalancer:
      ribbon:
        enabled: false # Ribbon is deprecated
```

Default Strategy:

- Round Robin (default)
- Can be customized

Health Checks

Load balancer should consider instance health:

```
eureka:
  instance:
    health-check-url-path: /actuator/health
```

Health-aware Load Balancing:

- Only routes to healthy instances
- Automatically excludes unhealthy instances
- Re-includes instances when they recover

Circuit Breaking

Circuit breaker prevents cascading failures:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: CircuitBreaker
              args:
                name: orderServiceCircuitBreaker
                fallbackUri: forward:/fallback/orders
```

How it Works:

1. Monitors failure rate
2. Opens circuit when threshold exceeded
3. Routes to fallback
4. Attempts to close circuit after timeout

Retry Logic

Automatic retry on failure:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: Retry
              args:
                retries: 3
                statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
                backoff:
                  firstBackoff: 10ms
                  maxBackoff: 50ms
                  factor: 2
```

Our Implementation

Load Balancing Configuration:

```
spring:
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      routes:
        - id: order-service
          uri: lb://order-service # Load balanced
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
```

Benefits in Our Architecture:

- Automatic load balancing across service instances
- Integration with Eureka for service discovery
- No manual load balancer configuration
- Scales automatically with instances

7. Simplifying Client-Side Communication

Complexity Without Gateway

Client Must Handle:

- Multiple service URLs
- Service discovery
- Load balancing
- Retry logic
- Circuit breaking
- Authentication
- Error handling
- API versioning

Example Client Code (Complex):

```
class OrderClient {
  constructor() {
    this.services = {
      auth: 'http://localhost:8081',
      customer: 'http://localhost:8082',
      order: 'http://localhost:8083',
      payment: 'http://localhost:8084'
    };
  }

  async createOrder(orderData) {
    // 1. Authenticate
    const token = await this.authenticate();

    // 2. Get customer
    const customer = await this.getCustomer(orderData.customerId, token);

    // 3. Create order
    const order = await this.createOrderInternal(orderData, token);

    // 4. Process payment
    const payment = await this.processPayment(order, token);

    return { order, payment };
  }

  async authenticate() {
    // Implement authentication logic
    // Handle retries, errors, etc.
  }

  async getCustomer(id, token) {
    // Implement service discovery
    // Implement load balancing
    // Implement retry logic
    // Handle errors
  }

  async createOrderInternal(order, token) {
```

```
        // Implement service discovery
        // Implement load balancing
        // Implement retry logic
        // Handle errors
    }

    async processPayment(order, token) {
        // Implement service discovery
        // Implement load balancing
        // Implement retry logic
        // Handle errors
    }
}
```

Simplicity With Gateway

Client Only Needs:

- Single gateway URL
- Standard HTTP client
- Basic error handling

Example Client Code (Simple):

```
class OrderClient {
  constructor() {
    this.gatewayUrl = 'http://localhost:8080';
    this.token = localStorage.getItem('token');
  }

  async createOrder(orderData) {
    const response = await fetch(`${this.gatewayUrl}/api/orders`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${this.token}`
      },
      body: JSON.stringify(orderData)
    });

    if (!response.ok) {
      throw new Error('Failed to create order');
    }

    return response.json();
  }
}
```

Benefits of Simplified Communication

1. Reduced Client Complexity

- Less code to maintain
- Easier to understand
- Fewer bugs
- Better developer experience

2. Consistent API Interface

- Unified API surface
- Consistent error handling
- Standardized responses
- Predictable behavior

3. Easier Testing

- Single endpoint to test
- Mock gateway for testing
- Simplified integration tests
- Better test coverage

4. Better Performance

- Fewer network calls
- Gateway-level caching
- Request/response optimization
- Reduced latency

5. Improved Security

- Centralized authentication
- Consistent security policies
- Reduced attack surface
- Easier compliance

6. Easier Maintenance

- Single place to update
- Version management
- Deprecation handling
- Backward compatibility

API Aggregation

Gateway can aggregate multiple service calls into one:

Without Aggregation (Multiple Calls):

```
// Client makes multiple calls
const customer = await fetch('/api/customers/123');
const orders = await fetch('/api/orders?customerId=123');
const payments = await fetch('/api/payments?customerId=123');
```

With Aggregation (Single Call):

```
// Gateway aggregates calls
const dashboard = await fetch('/api/dashboard/123');
// Returns: { customer, orders, payments }
```

Implementation:

```
@Component
public class DashboardAggregator {

    @Autowired
    private CustomerClient customerClient;

    @Autowired
    private OrderClient orderClient;

    @Autowired
    private PaymentClient paymentClient;

    public Mono<DashboardDTO> getDashboard(Long customerId) {
        return Mono.zip(
            customerClient.getCustomer(customerId),
            orderClient.getOrdersByCustomer(customerId),
            paymentClient.getPaymentsByCustomer(customerId)
        ).map(tuple -> new DashboardDTO(
            tuple.getT1(),
            tuple.getT2(),
            tuple.getT3()
        ));
    }
}
```

Request Composition

Gateway can compose requests from multiple services:

Example: Order with Customer Details

```
- id: order-with-customer
  uri: lb://order-service
  predicates:
    - Path=/api/orders-with-customer/**
  filters:
    - StripPrefix=1
    - name: RequestComposition
      args:
        services: customer-service
```

Response Transformation

Gateway can transform responses:

Example: Format Conversion

```
filters:
  - name: ResponseTransformation
    args:
      from: json
      to: xml
```

Our Implementation

Simplified Client Communication:

```
# All services accessible through single gateway
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
        - id: customer-service
          uri: lb://customer-service
          predicates:
            - Path=/api/customers/**
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
```

Client Benefits:

- Single URL: `http://localhost:8080`
- Consistent paths: `/api/{service}/**`
- No service discovery needed
- No load balancing logic
- Simplified error handling

8. When an API Gateway is Essential

Scenarios Requiring API Gateway

1. Multiple Client Types

- Web applications
- Mobile applications (iOS, Android)
- IoT devices
- Third-party integrations

Why Essential:

- Different clients need different API formats
- Centralized place to handle client-specific logic
- Consistent API across all clients

2. Complex Microservices Architecture

- 10+ microservices
- Inter-service communication
- Service dependencies
- Complex routing rules

Why Essential:

- Manages complexity
- Provides abstraction layer
- Simplifies client communication
- Enables independent service evolution

3. Security Requirements

- Authentication and authorization
- Rate limiting
- IP whitelisting/blacklisting
- Request validation
- Compliance requirements (GDPR, HIPAA)

Why Essential:

- Centralized security enforcement
- Consistent security policies
- Easier auditing
- Compliance management

4. High Traffic Volumes

- Thousands of requests per second
- Peak traffic handling
- DDoS protection
- Rate limiting

Why Essential:

- Load balancing
- Rate limiting
- Caching
- Request optimization

5. API Versioning

- Multiple API versions
- Backward compatibility
- Gradual migration
- Deprecation management

Why Essential:

- Version routing
- Compatibility management
- Smooth transitions
- Client control

6. Cross-Cutting Concerns

- Logging and monitoring
- Request/response transformation
- API aggregation
- Request composition

Why Essential:

- Centralized implementation
- Consistent behavior
- Reduced duplication
- Easier maintenance

7. External API Exposure

- Public APIs
- Partner APIs
- Third-party integrations
- API monetization

Why Essential:

- API key management
- Usage tracking
- Billing integration
- Access control

8. Performance Optimization

- Caching
- Compression
- Request/response optimization
- CDN integration

Why Essential:

- Improved performance
- Reduced latency
- Better user experience
- Cost optimization

When API Gateway Might Not Be Essential

1. Simple Monolithic Application

- Single service
- Few endpoints
- Simple routing
- Low traffic

Alternative:

- Load balancer (NGINX, HAProxy)
- Direct service access

2. Internal Microservices with Few Clients

- 2-3 microservices
- Internal use only
- Simple routing
- Low security requirements

Alternative:

- Service mesh (Istio, Linkerd)
- Direct service-to-service communication

3. Development/Testing Environment

- Local development
- Testing scenarios
- Proof of concept

Alternative:

- Direct service access
- Mock services
- Service stubs

Decision Framework

Use API Gateway if:

-  Multiple microservices (>5)
-  Multiple client types
-  Security requirements
-  High traffic volume
-  API versioning needed
-  Cross-cutting concerns
-  External API exposure
-  Performance optimization needed

Consider Alternatives if:

-  Single monolithic application
-  Few microservices (<3)
-  Internal use only
-  Low traffic volume
-  Simple routing
-  Development/testing only

Our Use Case

Food Delivery Platform:

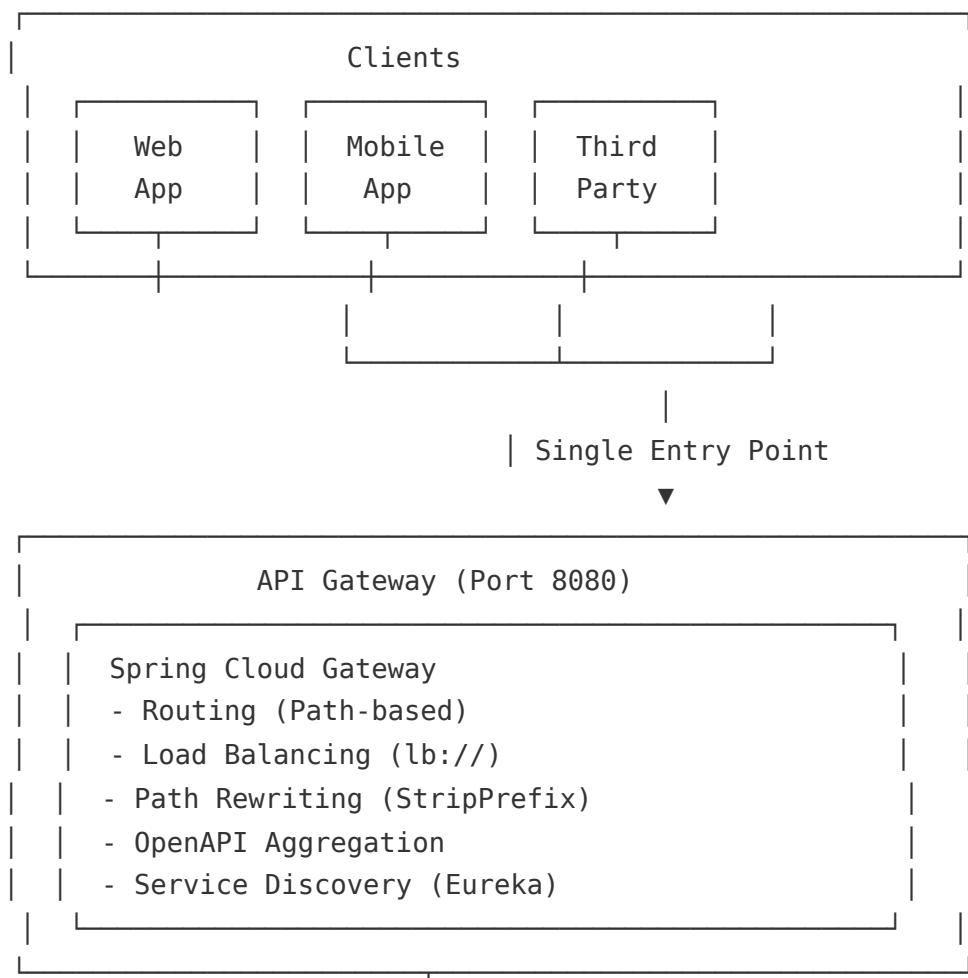
- 8 microservices
- Web and mobile clients
- Security requirements (JWT auth)
- High traffic potential
- API versioning needed
- Cross-cutting concerns (logging, monitoring)
- Performance optimization needed

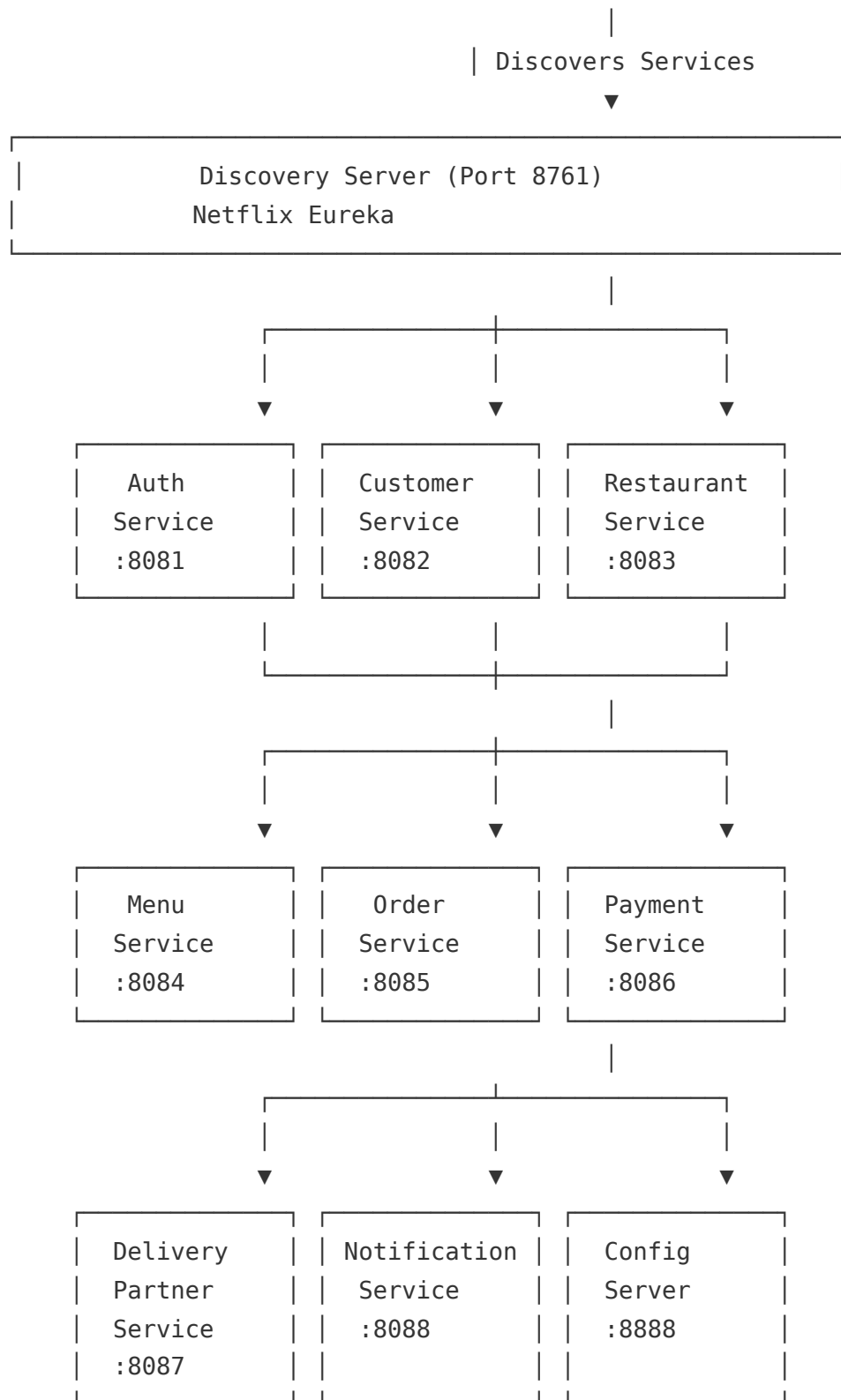
Conclusion:

API Gateway is **essential** for our Food Delivery Platform.

9. Real-world Architecture Example

Our Food Delivery Platform Architecture





Gateway Configuration Details

Application Configuration:

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  config:
    import: optional:configserver:http://localhost:8888
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
```

Service Routes:

```
spring:
  cloud:
    gateway:
      routes:
        # Core Services
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=1

        - id: customer-service
          uri: lb://customer-service
          predicates:
            - Path=/api/customers/**
          filters:
            - StripPrefix=1

        - id: restaurant-service
          uri: lb://restaurant-service
          predicates:
            - Path=/api/restaurants/**
          filters:
            - StripPrefix=1

        - id: menu-service
          uri: lb://menu-service
          predicates:
            - Path=/api/menus/**
          filters:
            - StripPrefix=1

        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
```

- id: delivery-partner-service
uri: lb://delivery-partner-service
predicates:
 - Path=/api/delivery-partners/**filters:
 - StripPrefix=1

- id: payment-service
uri: lb://payment-service
predicates:
 - Path=/api/payments/**filters:
 - StripPrefix=1

- id: notification-service
uri: lb://notification-service
predicates:
 - Path=/api/notifications/**filters:
 - StripPrefix=1

OpenAPI Aggregation Routes:

```
# OpenAPI Documentation Routes
- id: openapi-auth-service
  uri: lb://auth-service
  predicates:
    - Path=/v3/api-docs/auth-service
  filters:
    - RewritePath=/v3/api-docs/auth-service, /v3/api-docs

- id: openapi-customer-service
  uri: lb://customer-service
  predicates:
    - Path=/v3/api-docs/customer-service
  filters:
    - RewritePath=/v3/api-docs/customer-service, /v3/api-docs

- id: openapi-order-service
  uri: lb://order-service
  predicates:
    - Path=/v3/api-docs/order-service
  filters:
    - RewritePath=/v3/api-docs/order-service, /v3/api-docs
```

Swagger UI Configuration:

```
springdoc:
  api-docs:
    enabled: true
  swagger-ui:
    enabled: true
    path: /swagger-ui.html
    try-it-out-enabled: true
  urls:
    - name: "1. Auth Service"
      url: /v3/api-docs/auth-service
    - name: "2. Customer Service"
      url: /v3/api-docs/customer-service
    - name: "3. Restaurant Service"
      url: /v3/api-docs/restaurant-service
    - name: "4. Menu Service"
      url: /v3/api-docs/menu-service
    - name: "5. Order Service (Feign)"
      url: /v3/api-docs/order-service
    - name: "6. Delivery Partner"
      url: /v3/api-docs/delivery-partner-service
    - name: "7. Payment Service"
      url: /v3/api-docs/payment-service
    - name: "8. Notification Service"
      url: /v3/api-docs/notification-service
```

Request Flow Example

Scenario: Customer places an order

1. Client Request:

```
POST http://localhost:8080/api/orders
Headers:
  Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
  Content-Type: application/json

Body:
{
  "customerId": 123,
  "restaurantId": 456,
  "items": [
    {"menuItem": 789, "quantity": 2}
  ],
  "deliveryAddress": "123 Main St"
}
```

2. Gateway Processing:

1. Receive request on port 8080
2. Match route: Path=/api/orders/**
3. Apply filter: StripPrefix=1
4. Resolve service: lb://order-service via Eureka
5. Load balance: Select order-service instance
6. Forward request: POST http://order-service:8085/orders

3. Order Service Processing:

1. Receive request
2. Validate JWT token
3. Get customer details (via Feign client to customer-service)
4. Get menu items (via Feign client to menu-service)
5. Calculate total
6. Create order
7. Return response

4. Gateway Response:

HTTP/1.1 201 Created

Headers:

Content-Type: application/json

X-Gateway-Response: true

Body:

```
{  
  "orderId": 1001,  
  "status": "PENDING",  
  "total": 25.99,  
  "createdAt": "2024-01-15T10:30:00Z"  
}
```

Benefits in Our Architecture

1. Single Entry Point

- All clients access through `localhost:8080`
- Consistent API structure: `/api/{service}/**`
- Simplified client configuration

2. Centralized Routing

- All routing rules in one place
- Easy to add/remove services
- Dynamic route updates via Config Server

3. Service Discovery Integration

- Automatic service resolution via Eureka
- No hardcoded service URLs
- Dynamic instance discovery

4. Load Balancing

- Automatic load balancing with `lb://`
- Spring Cloud Load Balancer integration
- Fault tolerance and failover

5. Path Rewriting

- StripPrefix=1 removes /api prefix
- Clean service URLs
- Consistent path structure

6. OpenAPI Aggregation

- Single Swagger UI for all services
- Centralized API documentation
- Easy API testing

7. Monitoring

- Actuator endpoints for health and metrics
- Gateway-specific metrics
- Centralized logging

Docker Deployment

Docker Compose Configuration:

```
version: '3.8'
services:
  api-gateway:
    image: food-delivery/api-gateway
    ports:
      - "8080:8080"
    depends_on:
      - discovery-server
      - config-server
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE=http://discovery-server:8761/eu
      - SPRING_CLOUD_CONFIG_URI=http://config-server:8888
```

Docker Configuration:

```
# config-repo/application-docker.yml
eureka:
  client:
    service-url:
      defaultZone: http://discovery-server:8761/eureka/
```

Scaling the Gateway

Scale Gateway Instances:

```
docker-compose up --scale api-gateway=3
```

Benefits:

- High availability
- Load balancing at gateway level
- Fault tolerance
- Handle increased traffic

Monitoring and Observability

Actuator Endpoints:

- Health: `http://localhost:8080/actuator/health`
- Info: `http://localhost:8080/actuator/info`
- Gateway Routes: `http://localhost:8080/actuator/gateway/routes`

Metrics:

- Request count
- Response time
- Error rate
- Service-specific metrics

Security Enhancements (Future)

Potential Additions:

```
# JWT Authentication Filter
filters:
  - name: JwtAuthentication
    args:
      secret: ${JWT_SECRET}

# Rate Limiting
filters:
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 100
      redis-rate-limiter.burstCapacity: 200

# Circuit Breaker
filters:
  - name: CircuitBreaker
    args:
      name: orderServiceCircuitBreaker
      fallbackUri: forward:/fallback/orders

# Request Logging
filters:
  - name: RequestLogging
    args:
      includeHeaders: true
      includeBody: true
```

Conclusion

API Gateway is a critical component in microservices architecture that addresses the challenges of direct client access to multiple services. Spring Cloud Gateway provides a powerful, flexible solution for implementing API Gateway patterns.

Key Takeaways

1. **Single Entry Point:** Simplifies client configuration and reduces complexity
2. **Centralized Routing:** All routing logic in one place, easy to manage

3. **Security Enforcement:** Consistent security policies across all services
4. **Request Filtering:** Modify requests/responses before/after routing
5. **Load Balancing:** Automatic load balancing with service discovery
6. **Simplified Communication:** Clients only need to know gateway URL
7. **Cross-Cutting Concerns:** Centralized implementation of logging, monitoring, etc.

Best Practices

1. Use service discovery integration (Eureka)
2. Implement proper path rewriting
3. Add authentication and authorization filters
4. Implement rate limiting for production
5. Use circuit breakers for fault tolerance
6. Monitor gateway metrics and logs
7. Implement proper error handling
8. Use OpenAPI aggregation for documentation

Our Food Delivery Platform

Our implementation demonstrates:

- Spring Cloud Gateway with service discovery
- Centralized routing for 8 microservices
- Path rewriting with StripPrefix
- Load balancing with lb:// URIs
- OpenAPI aggregation for unified documentation
- Docker deployment support
- Monitoring with Actuator

This architecture provides a solid foundation for a scalable, secure, and maintainable microservices system.

References

- [Spring Cloud Gateway Documentation](#)
- [Spring Cloud Load Balancer](#)

- [API Gateway Pattern](#)
- [Backend for Frontend Pattern](#)

Document generated for Food Delivery Platform Microservices Architecture